cs5800-hw_1_exercises

Mingxi Zhu

May 2026

1 Exercise 1.20

Find the inverse of: 20 mod 79, 3 mod 62, 21 mod 91, 5 mod 23. Explain which algorithm to use and why.

Answer:
Use Extended Euclid Algorithm. The inverse of x modulo y calculation can be the multiplicative a in Bezout's Equation $ax + by = gcd(x, y)$

```
def extEuclid(x, y):
    if y == 0:
        return 1, 0, x
    a, b, d = extEuclid(y, x % y)
    return b, a-(x//y)*b, d

a1, _, _ = extEuclid(20, 79) % 79
a2, _, _ = extEuclid(3, 62) % 62
a3, _, _ = extEuclid(21, 91) % 91
a4, _, _ = extEuclid(5, 23) % 23
```

a1 = 4
a2 = 21 a3 = 87 a4 = 14

2 Exercise 1.24

If p is prime, how many elements of {0, 1, ..., p^n - 1}have an inverse modulo p^n?

Answer:
$p^n - n$
because p is prime, any $p^k (0 \leq k < n)$ has $gcd(p^k, p)! = 1$. There are $n - 1$ elements of p^k. Plus, 0 doesn't have an inverse modulo with any number. Therefore, total n elements don't have an inverse modulo p^n.

3 Exercise 1.25

Calculate 2^{125} mod 127 using any method you choose. (Hint: 127 is prime.) Explain your methodology.

Answer:
Use fast exponent with a modulo of 127 to calculate it.

```
def fastExp(x, e, N):
    if e == 0:
        return 1
    if e == 1:
        return x
    rem = e & 1
    sp = fastExp(x, e>>1, N)
    ans = sp * sp % N
    if rem == 1:
        ans = ans * x % N
    return ans

ans = fastExp(2, 125, 127)
```

ans = 64

4 Exercise ex1

Given the prime numbers p = 823 and q = 173, what are the two public keys (N, e) and the private key (d)? In addition, show the encrypted version of the message 42.

Answer:
Public key: (N, e) = (142379, 52013) Private key: d = 53669 Encrypted msg: 50804

```
def main(p, q, msg):
    N = p * q
    phi = (p-1)*(q-1)
    e = random.randint(2, phi-1)
    while gcd(e, phi) != 1:
        e = random.randint(2, phi-1)
    d, _, _ = extEuclid(e, phi)
    d %= phi
    encrypted = fastExp(msg, e, N)
```

```
    return N, e, d, encrypted

N, e, d, encrypted = main(823, 173, 42)
```

5 Exercise ex2

Use the divide and conquer integer multiplication algorithm to multiply the two binary integers 1011 and 0110. (Figure 2.1 in Dasgupta et. al.)

Answer:

```
def fastMul(x, y):
    n = max(size of x, size of y)
    if n == 1:
        return x*y
    half = (n+1) // 2
    mask = 1 << half - 1
    xl, xr = x >> half, x & mask
    yl, yr = y >> half, y & mask
    p1 = fastMul(xl, yl)
    p2 = fastMul(xr, yr)
    p3 = fastMul(xl+xr, yl+yr)
    return p1<<(2*half)+(p3-p1-p2)<<half+p2

ans = fastMul(1011, 0110)
```

$ans = 1000010_2$

6 Exercise A3.Problem 3

From the Goddard text, section A3 problem 3. What is the big-O complexity of your algorithm:
Suppose we have a list of n numbers. The list is guaranteed to have a number which appears more than n/2 times on it. Devise a good algorithm to find the Majority element.

Answer:
Linear traversal. $T(f) = O(n)$

```
def findMajor(list):
    counter = 1
```

```
ans = list[0]
for i in range(1, len(list)):
    if list[i] == ans:
        counter += 1
    else:
        counter -= 1
    if counter == 0:
        ans = list[i]
return ans
```

Based on Master Theorem, using divide and conquer algorithm is have to traverse the merged interval again to know the times of the majority number. will make f(n) closer to O(n), making the complexity closer to $O(nlogn)$

7 LLM Disclosure

1. study how to create a mask for binary integers 2. study why modulo can be calculated regardless of sequence 3. revise why e in RSA needs to be in between 1 and phi